

Free Form Testing Agent

Document: 03_state_model.md

Version: 1.0

Status: Draft

Author: Kalpesh

Date: June 2026

1. Purpose

The State Model is the foundation of the Free Form Testing Agent (FFTA).

Every application interaction performed by the agent transforms the application from one state to another. The purpose of the state model is to:

- Represent application behavior.
- Enable autonomous exploration.
- Prevent repeated exploration.
- Build workflow graphs.
- Reproduce bugs.
- Generate test cases.
- Measure coverage.

The entire application is modeled as a directed graph:

``text id="l2gq8s" State + Action = Next State

2. State Modeling Philosophy

Traditional automation thinks in terms of:

``text id="6sq9xq"

Test Case

↓

Step

↓

Assertion

The Free Form Testing Agent thinks in terms of:

```text id="o8bqeu" State ↓ Action ↓ State

Example:

```text id="gr5jv8"

State A:

Display = 2

Action:

+

State B:

Display = 2+

3. State Definition

A state represents a complete snapshot of the application at a particular moment.

A state consists of:

- UI hierarchy
- Window metadata
- Visible controls
- Control properties
- Application values
- Screenshot
- Available actions
- Runtime metadata

4. State Object

```python id="dny9v4" class ApplicationState:

```
state_id: str
```

```
timestamp: datetime
```

```
window_title: str
```

```
controls: list
values: dict
available_actions: list
screenshot_path: str
metadata: dict
```

'''

---

## 5. State Components

### 5.1 Window Information

Stores application-level information. Example: `json id="4zlx0u"`

```
{
 "title": "Calculator",
 "class": "ApplicationFrameWindow",
 "width": 800,
 "height": 600,
 "is_active": true
}
```

---

### 5.2 Control Tree

Represents visible controls.

Example:

```
```json id="lyog2e" [{ "id": "btn_1", "type": "Button", "name": "1" }, { "id": "btn_plus", "type": "Button",
"name": "+" }, { "id": "display", "type": "Text", "value": "12" } ]
```

```
---
```

5.3 Runtime Values

Stores current application values.

Example:

```
```json id="3v1w8m"
{
 "display": "100",
 "operator": "+",
 "memory": "0"
}
```

---

## 5.4 Screenshot

Stores visual representation.

Example:

```text id="d7yrvn" storage/screenshots/

state_0001.png state_0002.png state_0003.png

```
---

## 5.5 Available Actions

Actions possible from this state.

Example:

```json id="b4zk0n"
[
 "0",
 "1",
 "2",
 "+",
 "-",
 "*",
 "/",
 "=",
 "C"
]
```

## 6. State Identifier

Every state receives a unique identifier.

Example:

```
```text id="4vbh8w" state_000001 state_000002 state_000003
```

```
---  
  
## UUID Format  
  
```python id="t2rlab"  
uuid4()
```

Example:

```
```text id="t8x5uk" 550e8400-e29b-41d4
```

```
---  
  
# 7. State Hashing  
  
The agent must determine whether two states are identical.  
  
A hash is generated using:  
  
```text id="jlmvjv"  
Window
+
Controls
+
Values
+
Metadata
```

---

## Example

State:

```
```json id="4u8juh" { "display": "2", "operator": "+" }
```

Hash:

```
```text id="m3gmbe"
ab6b37d1ef
```

---

## Hash Function

```
```python id="jbhz0m" import hashlib
```

```
def create_hash(data):
```

```
    return hashlib.sha256(
        data.encode()
    ).hexdigest()
```

```
```
```

---

## 8. State Equality

Two states are equal when: `text id="mj1wmj"`  
`hash(state_a) == hash(state_b)`

Example:

```
```text id="a8bbmg" 2 + 2
```

equals

2 + 2

```
---
```

```
# 9. State Similarity
```

```
Sometimes states are not identical but similar.
```

```
Example:
```

```
```text id="akos2n"
Display=10
```

Display=11

Similarity:

```text id="dhyhgo" 95%

```
---  
  
## Similarity Algorithm  
  
Possible approaches:  
  
### Exact Match  
  
```text id="kcc0jl"  
100%
```

---

## Structural Match

```text id="65nql2" Compare control trees.

```
---  
  
### Visual Match  
  
```text id="7fjlmw"  
Compare screenshots.
```

---

## Semantic Match

```text id="e4obww" Use LLM reasoning.

```
---  
  
# 10. State Metadata  
  
Metadata provides debugging information.  
  
Example:
```

```
```json id="8gm83n"
{
 "session": "123",
 "depth": 4,
 "timestamp": "2026",
 "explorer": "BFS",
 "visited": true
}
```

---

## 11. Action Model

Actions transform states.

---

### Action Object

```python id="hh4doq" class Action:

```
    action_id:str

    action_type:str

    target:str

    value:str

    timestamp:str
```

```

---

### Example

```
json id="edjj1w"
{
 "action": "CLICK",
 "target": "btn_7"
}
```

---



## 12. Action Types

Supported actions:

```
```text id="f88m8m" CLICK DOUBLE_CLICK KEY_PRESS TEXT_INPUT SCROLL DRAG HOTKEY WAIT
```

```
---  
  
# 13. State Transition  
  
A transition represents:  
  
```text id="qz4hbx"  
State A
 +
Action
 =
State B
```

Example:

```
```text id="n4v6yk" 0
```

Click(7)

7

```
---  
  
## Transition Object  
  
```python id="tdun5u"  
class Transition:

 source_state

 action

 target_state

 duration
```

success

---

## 14. Transition Graph

The application is modeled as:

```
```text id="lllv1n" S0 /  
/  
A1 A2 | | V V S1 S2 | | A3 | V S3
```

```
---  
  
# 15. State Graph Object  
  
```python id="7fh9lm"  
class StateGraph:

 states

 transitions

 visited

 coverage
```

---

## 16. State Storage

States are persisted.

---

### Database Schema

```
```sql id="vl0qvp" states
```

id hash title metadata screenshot timestamp

```
---

## Transition Table

```sql id="9ik0u7"
transitions

source
target
action
duration
success
```

---

## 17. State Exploration

The graph supports:

### Breadth First Search

```text id="q4vtpq" Explore nearby states.

```
---

## Depth First Search

```text id="2w1ccf"
Explore deeply.
```

---

### Random Search

```text id="wnkzba" Explore randomly.

```
---

## AI Guided Search

```text id="7mkuyf"
Use reasoning.
```

## 18. State Coverage

Coverage metrics:

---

### State Coverage

```text id="o4mdlB" Visited states

/ Total states

```
---  
  
## Action Coverage  
  
```text id="t5nsyu"  
Executed actions

/
Available actions
```

---

### Transition Coverage

```text id="kvwzcy" Executed transitions

/ Possible transitions

```
---  
  
# 19. State Replay  
  
Every state sequence can be replayed.  
  
Example:  
  
```text id="wms95f"  
S0
↓
7
↓
S1
↓
```

```
/
↓
S2
↓
0
↓
S3
↓
=
↓
Crash
```

---

## 20. State Compression

Large applications may generate millions of states.

Compression strategies:

---

### Remove duplicates

```text id="m0tqjw" Keep only unique states.

```
---

## Merge similar states

```text id="kmpzmw"
Similarity threshold.
```

---

### Prune dead branches

```text id="atq1ru" Remove unreachable states.

```
---

# 21. State Learning

The agent learns:
```

```
```json id="k5qmtc"
{
 "pattern":"division_by_zero",
 "states":[
 "S1",
 "S2",
 "S3"
]
}
```

---

## 22. Calculator Example

Initial state:

```text id="zjk80r" Display: 0

State:

```
```json id="h4l2zv"
{
 "id":"S0",
 "display":"0",
 "actions":[
 "0",
 "1",
 "2",
 "+",
 "-"
]
}
```

Action:

```text id="t3tyxo" Click 7

New state:

```
```json id="ggn1h0"
{
```

```
"id": "S1",
"display": "7"
}
```

Action:

```text id="hyah6h" Click /

```
---  
  
New state:  
  
```json id="83wymq"  
{
 "id": "S2",
 "display": "7/"
}
```

Action:

```text id="gh7ixd" Click 0

```
---  
  
New state:  
  
```json id="vjlmw8"  
{
 "id": "S3",
 "display": "7/0"
}
```

Action:

```text id="3h59tq" Click =

```
---  
  
Result:
```

```
```text id="h0tx5h"
Error State
```

Graph:

```
```text id="98qmmr" S0 | 7 | S1 | / | S2 | 0 | S3 | = | Error
```

```
---

# 23. Future State Models

Future versions will support:

---

## Visual States

```text id="j4vq3f"
Screenshot embeddings.
```

## Semantic States

```
```text id="6o50j2" LLM understanding.
```

```
---

## Behavioral States

```text id="ztmb1q"
Workflow patterns.
```

## Knowledge Graph States

```
```text id="ys1b3n" Entity relationships.
```

```
---
```


24. Design Decisions

Decision	Choice
-----	-----
State Model	Graph
Storage	SQLite
Graph Engine	NetworkX
Hashing	SHA256
Replay	Supported
Similarity	Pluggable
Screenshots	PNG
State IDs	UUID

25. Conclusion

The state model is the core abstraction of the Free Form Testing Agent.

Everything in the system eventually becomes:

```
```text id="g0y4fh"
Observe State
 ↓
Store State
 ↓
Execute Action
 ↓
Observe New State
 ↓
Create Transition
 ↓
Build Graph
```

This graph representation enables:

- Autonomous exploration
- Coverage measurement
- Bug reproduction
- Workflow reconstruction
- Test generation
- Agent learning
- Future AI reasoning